HANIKA ANGLICAN INTEGRATED POLYTECHNIC (HAIP) · Data Mining and Data Warehousing · FA1 Group Work

Design Document

Architecture, dimensional model, ETL design and justifications

---

Project: University Library Analytics Data Warehouse

Group: **Group X** (replace with your group number)

Members: Member 1 (Data Architect) · Member 2 (ETL Specialist) · Member 3 (Analytics & Security Lead)

Database: SQLite 3.53.1 · ETL: Python 3 + pandas · Dashboards: Power BI Desktop

Submission: November 2, 2026 · Presentation: November 6, 2026

Contents

1. Introduction
   - 1.1 The problem
   - 1.2 Objectives
   - 1.3 Scope and assumptions
   - 1.4 Results in numbers
2. Approach
   - 2.1 Kimball (bottom-up) instead of Inmon (top-down)
   - 2.2 ETL instead of ELT
3. Architecture
   - 3.1 Staging area structure
4. Technology choices
5. Dimensional model
   - 5.1 Star vs snowflake vs galaxy
   - 5.2 Fact tables
   - 5.3 Dimension tables
   - 5.4 Key design decisions
6. Entity-Relationship Diagram
7. Data dictionary
   - 7.1 Fact tables
   - 7.2 Dimension tables
   - 7.3 Staging, control and security tables
8. ETL design
   - 8.1 Extraction plan
   - 8.2 Transformation logic
   - 8.3 Loading strategy
   - 8.4 Error handling
   - 8.5 Data refresh schedule
9. Indexes and performance
10. Security, backup and OLAP (summary)
11. Scalability
12. Answers to the stakeholder questions
13. Project files and how to run

1. Introduction

1.1 The problem

The University Library runs three separate legacy systems that do not talk to each other:

System	Technology	Records
Book Lending System	MySQL database (`book_transactions`)	book checkouts and returns
Digital Resources System	Excel workbook (`digital_usage.xlsx`)	e-book, journal and article downloads
Study Room Booking System	web application with CSV exports (`room_bookings.csv`)	study-room reservations

Every month the Library Director copies data from the three systems by hand. This takes more than 15 hours and the reports are inconsistent, because the systems write the same things in different ways ("CS", "Computer Science", "CompSci"; "R101", "R-101", "Room101"; three date formats). There is no history, no trend analysis and no dashboard.

1.2 Objectives

We designed and built a **data warehouse** that:

1. **consolidates** the three sources (plus two reference files) into one database;
2. **cleans and standardises** the data automatically with an ETL process, logging every problem;
3. **organises** the data in a dimensional model (galaxy schema of three star schemas);
4. **enables OLAP** operations: drill-down, roll-up, slice, dice and pivot;
5. **provides dashboards** in Power BI (Executive, Department, Operational);
6. **secures** the data with role-based access control, masking, row-level security and audit logging;
7. **ensures data quality** with validation rules, before/after metrics and an error log;
8. **plans backup, recovery and scalability**.

1.3 Scope and assumptions

- **Sample data.** No real library data was supplied, so `09_Source_Data/generate_source_data.py` simulates the three systems for **2023-2025**: 800 students, 72 books, 46 digital resources and 11 study rooms. The data is made *dirty on purpose* with every issue listed in section 7 of the brief (mixed date formats, inconsistent names, NULLs, exact/logical/near duplicates, numbers stored as text). A fixed random seed makes the data identical on every computer.
- **Digital source has no StudentID.** It records the user *type* and faculty, not the person. Analyses per student are therefore only possible for books and rooms (see question 11).
- **ResourceTitle** is assumed to be exported by the digital system; without it "Top 10 downloaded journals" could not be answered.
- **Reference data**: a student master list (department, level) and a book catalogue (titles, categories) are assumed to be available from the Registrar and the catalogue system.
- **Loan period** is 21 days: a loan kept longer is *overdue*.

1.4 Results in numbers

9,390	9,065	527	0	~5 s
raw source rows extracted	clean rows loaded	problems logged and handled	duplicates in the warehouse	full rebuild (was 15+ hours)

2. Approach

2.1 Kimball (bottom-up) instead of Inmon (top-down)

	Inmon (top-down)	Kimball (bottom-up) - chosen
Starts with	an enterprise-wide normalised (3NF) warehouse, data marts later	dimensional data marts built around business processes
Time to first result	long	short
Best for	large organisations, many subject areas	one department with clear questions
Users	IT specialists	business users, BI tools

Why Kimball: the library is one subject area with a fixed list of business questions and a deadline of 1.5 weeks. Kimball's dimensional model is easy to understand for non-technical users and is exactly the shape Power BI expects. Each library service (loans, downloads, bookings) is a *business process* with its own fact table; **conformed dimensions** (date, department, student) connect them - Kimball's "bus architecture". Each fact table with its dimensions is a small **data mart** (Circulation, Digital, Rooms), and the department views of the security layer are department-level marts.

2.2 ETL instead of ELT

	ETL - transform before load (**chosen**)	ELT - load then transform in the database
Where the cleaning happens	Python (pandas)	SQL inside the database
Good for	complex text cleaning, small/medium data	very large data on powerful databases

The cleaning rules (date parsing with several formats, fuzzy duplicates, text-to-number conversion) are easier to write, read and test in Python. The data volume is small. Raw data is still kept in the **staging tables**, so the cleaning can always be re-run and audited.

3. Architecture

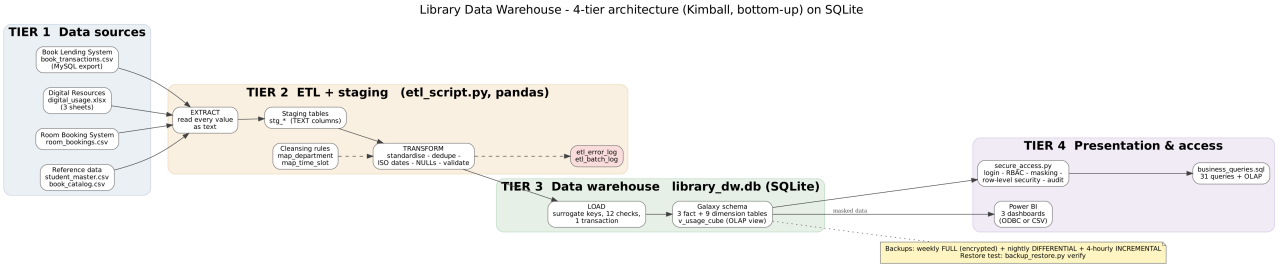


Figure 1 - 4-tier architecture

Tier	Components	Purpose
1. Sources	`book_transactions.csv`, `digital_usage.xlsx` (3 sheets), `room_bookings.csv`, `reference/student_master.csv`, `reference/book_catalog.csv`	the original systems' exports, never changed
2. ETL + staging	`etl_script.py`; tables `stg_*` (raw text), `map_*` (cleansing rules), `etl_error_log`, `etl_batch_log`	extract, clean, validate, log
3. Warehouse	`library_dw.db`: 3 fact tables, 9 dimensions, `v_usage_cube`	the single source of truth
4. Presentation	`secure_access.py`, `business_queries.sql`, Power BI (3 dashboards)	secure analysis and reporting

3.1 Staging area structure

- Every source column is stored as **TEXT**, exactly as received. A value such as "2 hrs" or "31-Feb-2024" can never make the load fail.
- Audit columns on every staging row: `batch_id` (which ETL run), `source_file`, `row_num` (original line number, so every error can be traced back to the file) and `loaded_at`.
- **Cleansing rules are data**: `map_department` (27 spellings → 8 codes) and `map_time_slot` (21 spellings → 6 blocks) are read by the ETL at start-up. A new spelling is fixed by inserting one row - no code change.
- `etl_error_log` stores every rejected, corrected or defaulted value with the reason; `etl_batch_log` stores one row per run (rows extracted / loaded / rejected, status).

4. Technology choices

Layer	Tool	Why
Database	**SQLite 3.53.1**	listed in the brief; no server to install, the whole warehouse is one portable file (2,976 KB) that every team member and the examiner can open; supports everything we need: foreign keys, CHECK constraints, indexes, views, triggers, recursive CTEs and window functions
ETL	Python 3 + pandas	recommended in the brief; flexible text cleaning; SQLite is built into Python
Analysis	SQL (`business_queries.sql`) + Python	31 queries, bonus analytics
Dashboards	Power BI Desktop	free, professional; reads SQLite via ODBC or the exported CSV files
GUI for the database	DB Browser for SQLite (free)	browse tables and run queries

Limitations of SQLite and how we handle them

Limitation	Our solution
no user accounts / GRANT	security tables + `secure_access.py` with SQLite's authorizer (section 10)
one writer at a time	only the nightly ETL writes; everyone else reads
no built-in encryption	encrypted backups (AES), BitLocker on the server disk; SQLCipher possible
not a network server	for many concurrent users, migrate to MySQL/PostgreSQL (section 11)

5. Dimensional model

5.1 Star vs snowflake vs galaxy

Schema	Description	Verdict
Star	one fact table, denormalised dimensions	good, but one fact table cannot hold three different processes cleanly
Snowflake	dimensions split into sub-tables (normalised)	more joins, slower, harder in Power BI → **rejected**
Galaxy (fact constellation)	several fact tables sharing dimensions	**chosen** - each fact table is itself a clean star

Justification. The three services are measured differently and at different grains (loan duration, number of downloads, booking hours). The suggested single `library_usage_fact` would be mostly NULL columns. With three facts sharing **conformed** `dim_date` and `dim_department` we can still compare services in one report (e.g. "books vs downloads vs rooms per month"). For questions that need all services in one result, the view `v_usage_cube` stacks the three facts on the shared dimensions (date, department, service). All dimensions are kept *denormalised* (one level), which gives the speed and simplicity of a star schema.

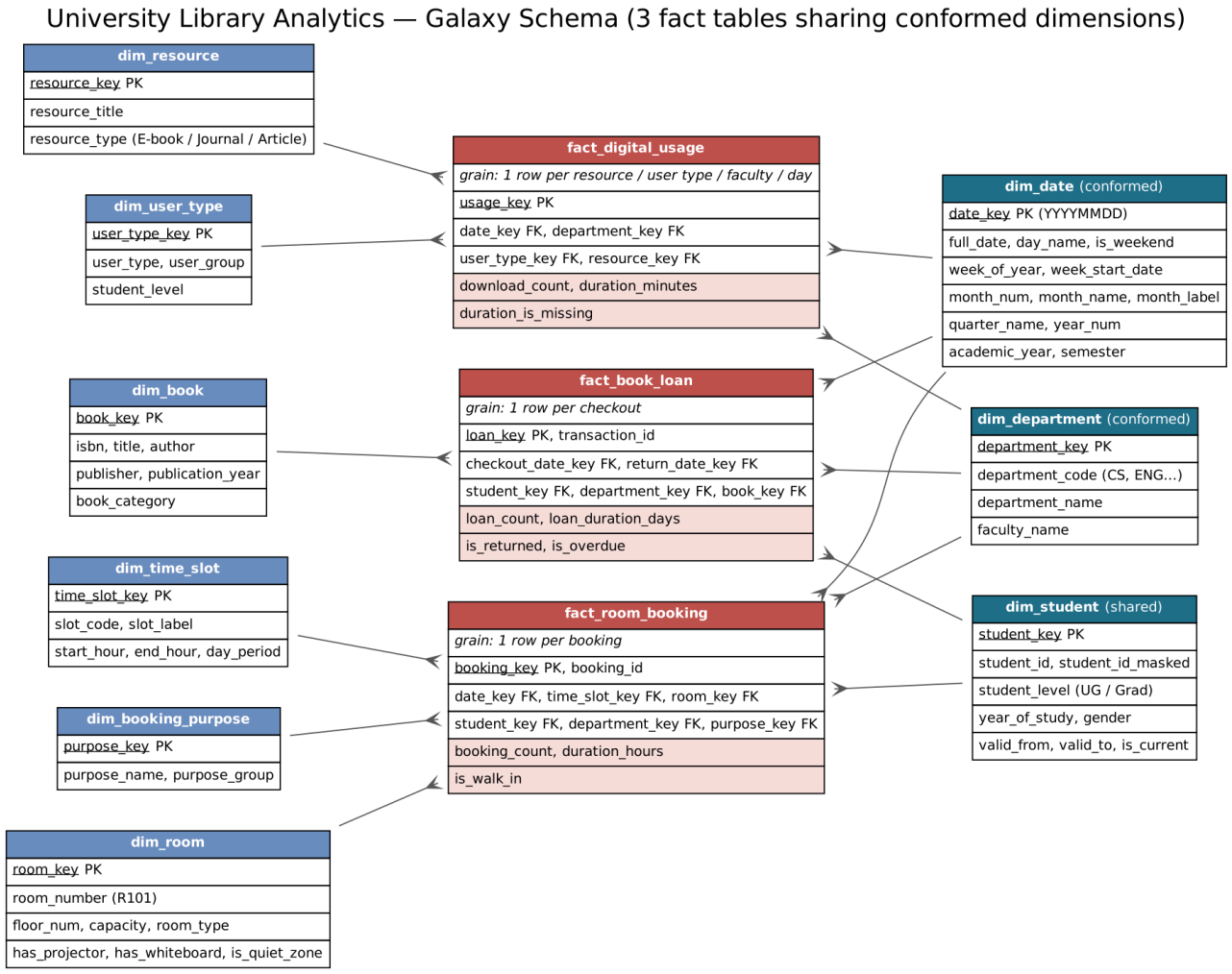


Figure 2 - Galaxy schema: three fact tables (red) sharing conformed dimensions (dark blue)

5.2 Fact tables

Fact table	Grain (one row =)	Measures	Dimension keys	Rows
`fact_book_loan`	one book checkout	`loan_count` (=1), `loan_duration_days`, `is_returned`, `is_overdue`	checkout date, return date, student, department, book	2,205
`fact_digital_usage`	downloads of one resource by one user type of one faculty on one day	`download_count`, `duration_minutes`, `duration_is_missing`	date, department, user type, resource	4,023
`fact_room_booking`	one booking	`booking_count` (=1), `duration_hours`, `is_walk_in`	date, time slot, room, student, department, purpose	2,837

`transaction_id` and `booking_id` are kept as *degenerate dimensions* (source IDs) with a **UNIQUE** constraint, so a record can never be loaded twice. `fact_room_booking` also has UNIQUE(room, date, time slot): a room cannot be double-booked.

5.3 Dimension tables

Dimension	Key	Main attributes	Notes
`dim_date`	`date_key` (YYYYMMDD)	day, day name, weekend, week, week start, month, quarter, year, academic year, semester	2023-2027, generated by a recursive SQL query; row −1 = "not returned"
`dim_department`	`department_key`	code, name, faculty	conformed; 8 departments + Unknown
`dim_student`	`student_key`	student_id, **student_id_masked**, level, year of study, gender, home department, valid_from/valid_to/is_current	shared by loans and rooms; SCD type 2 columns
`dim_book`	`book_key`	ISBN, title, author, publisher, year, category	from the catalogue
`dim_resource`	`resource_key`	title, type (E-book / Journal / Article)	built from the digital data
`dim_user_type`	`user_type_key`	user type, group, student level	lets us compare undergraduates and graduates for downloads
`dim_room`	`room_key`	room number, floor, capacity, type, projector, whiteboard, quiet zone	answers "why are rooms popular?"
`dim_time_slot`	`time_slot_key`	code, label, start/end hour, part of day	6 standard 2-hour blocks
`dim_booking_purpose`	`purpose_key`	purpose, group (individual / collaborative)	

5.4 Key design decisions

Question from the brief	Decision
How to handle dates across systems?	The ETL converts every format to ISO 8601; every fact points to the one `dim_date` table through `date_key` = YYYYMMDD. All hierarchies (day → week → month → quarter → year, academic year, semester) live in `dim_date`.
Fact table(s)?	three: `fact_book_loan`, `fact_digital_usage`, `fact_room_booking`

Question from the brief	Decision
Dimensions?	time (`dim_date`), student, resource (`dim_book`, `dim_resource`), location (`dim_room`) + department, user type, time slot, purpose
One fact table or several?	several (galaxy) - section 5.1
Surrogate keys	integer keys created by the warehouse; the source IDs are kept as natural keys, so a change of ID format in a source does not break the warehouse
Missing values	each dimension has an **Unknown member (key 1)**; a walk-in booking points to the Unknown student instead of NULL; a book not returned has `return_date_key = -1`
History	`dim_student` has SCD type 2 columns for students who change department or level
Privacy	`student_id_masked` (STU-****-001) is the only ID analysts, heads and Power BI ever see
Data-quality guards	CHECK constraints (no negative counts or durations, hours 0-12, flags 0/1), foreign keys ON, UNIQUE keys

6. Entity-Relationship Diagram

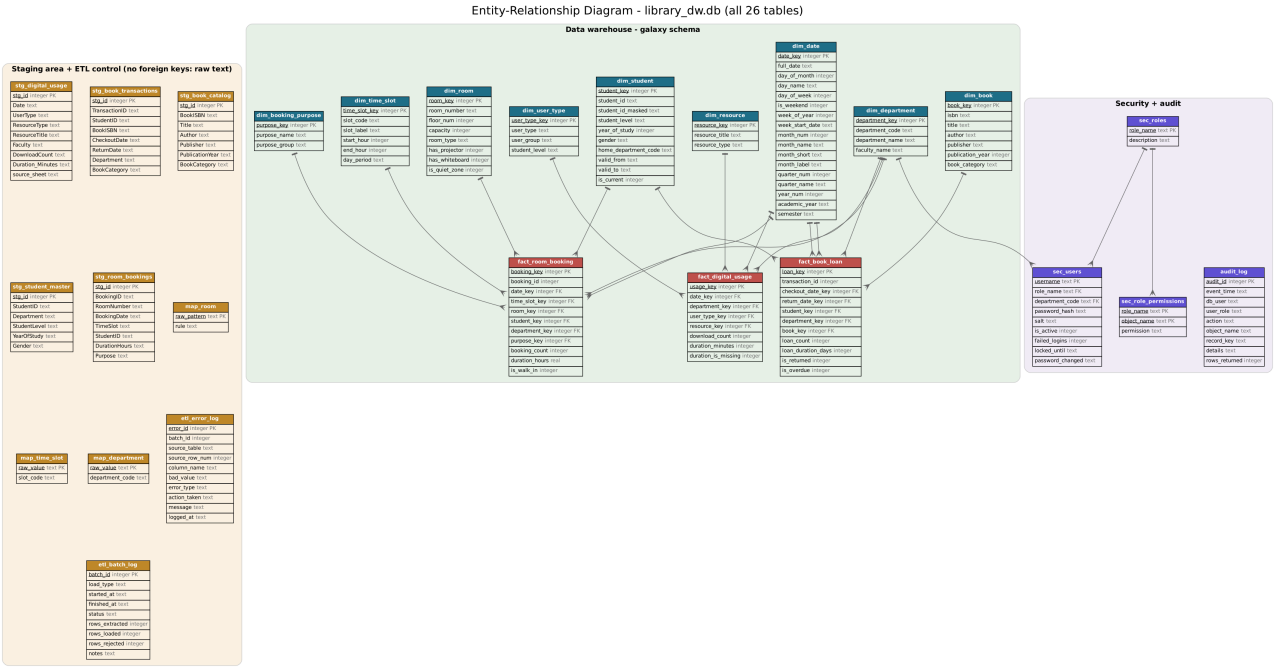


Figure 3 - All 26 tables of library_dw.db. Crow's foot = "many"; the staging tables have no foreign keys because they hold raw text.

Relationships (all one-to-many, dimension → fact): `dim_date` → the three facts (and `return_date_key` → `dim_date`); `dim_department` → the three facts and `sec_users`; `dim_student` → loans and bookings; `dim_book` → loans; `dim_resource`, `dim_user_type` → downloads; `dim_room`, `dim_time_slot`, `dim_booking_purpose` → bookings; `sec_roles` → `sec_users`, `sec_role_permissions`.

7. Data dictionary

7.1 Fact tables

fact_book_loan

Column	Type	Description
loan_key	INTEGER PK	surrogate key
transaction_id	INTEGER UNIQUE	TransactionID from the lending system (degenerate dimension)
checkout_date_key	INTEGER FK → dim_date	day borrowed
return_date_key	INTEGER FK → dim_date	day returned; −1 = not returned yet
student_key	INTEGER FK → dim_student	borrower (1 = Unknown)
department_key	INTEGER FK → dim_department	standardised department
book_key	INTEGER FK → dim_book	book (1 = Unknown Book)
loan_count	INTEGER	always 1; SUM = number of loans
loan_duration_days	INTEGER, ≥ 0	return − checkout; NULL if not returned
is_returned	INTEGER 0/1	1 = returned
is_overdue	INTEGER 0/1	1 = kept longer than 21 days
etl_batch_id, loaded_at		audit: which ETL run, when

fact_digital_usage

Column	Type	Description
usage_key	INTEGER PK	surrogate key
date_key	INTEGER FK → dim_date	download day
department_key	INTEGER FK → dim_department	faculty of the users
user_type_key	INTEGER FK → dim_user_type	undergraduate / graduate / faculty / staff
resource_key	INTEGER FK → dim_resource	resource downloaded
download_count	INTEGER ≥ 0	number of downloads
duration_minutes	INTEGER, nullable	reading time; NULL for legacy records
duration_is_missing	INTEGER 0/1	1 = duration was missing in the source
etl_batch_id, loaded_at		audit

fact_room_booking

Column	Type	Description
booking_key	INTEGER PK	surrogate key
booking_id	INTEGER UNIQUE	BookingID from the booking system
date_key	INTEGER FK → dim_date	booking day
time_slot_key	INTEGER FK → dim_time_slot	standard 2-hour block
room_key	INTEGER FK → dim_room	standardised room
student_key	INTEGER FK → dim_student	1 = Unknown (walk-in)
department_key	INTEGER FK → dim_department	department of the student
purpose_key	INTEGER FK → dim_booking_purpose	reason
booking_count	INTEGER	always 1
duration_hours	REAL, 0-12	length of the booking
is_walk_in	INTEGER 0/1	1 = no StudentID in the source
etl_batch_id, loaded_at		audit

7.2 Dimension tables

Table	Column	Type	Description
dim_date	date_key	INTEGER PK	YYYYMMDD, e.g. 20240115
	full_date	TEXT UNIQUE	ISO date 2024-01-15
	day_of_month, day_name, day_of_week, is_weekend		1 = Monday ... 7 = Sunday
	week_of_year, week_start_date		week number (weeks start on Monday) and its Monday
	month_num, month_name, month_short, month_label		1, January, Jan, 2024-01
	quarter_num, quarter_name, year_num		1, Q1, 2024
	academic_year, semester		2023/2024 (starts in September); Semester 1 / 2 / Break
dim_department	department_key, department_code, department_name, faculty_name		CS / Computer Science / Faculty of Computing & IT
dim_student	student_key	INTEGER PK	surrogate key
	student_id	TEXT	real ID - hidden from every non-admin role
	student_id_masked	TEXT	STU-****-001
	student_level, year_of_study, gender, home_department_code		Undergraduate / Graduate
	valid_from, valid_to, is_current		SCD type 2 history
dim_book	book_key, isbn, title, author, publisher, publication_year, book_category		category: Textbook, Research, Reference, Fiction
dim_resource	resource_key, resource_title, resource_type		E-book, Journal, Article
dim_user_type	user_type_key, user_type, user_group, student_level		
dim_room	room_key, room_number, floor_num, capacity, room_type, has_projector, has_whiteboard, is_quiet_zone		
dim_time_slot	time_slot_key, slot_code, slot_label, start_hour, end_hour, day_period		08-10 ... 18-20
dim_booking_purpose	purpose_key, purpose_name, purpose_group		Individual / Collaborative

7.3 Staging, control and security tables

Table	Purpose
stg_book_transactions, stg_digital_usage, stg_room_bookings	raw copy of each source (all TEXT) + batch_id, source_file, row_num, loaded_at
stg_student_master, stg_book_catalog	raw reference data
map_department, map_time_slot, map_room	cleansing rules stored as data
etl_batch_log	one row per ETL run: type, start/end, status, rows extracted/loaded/rejected
etl_error_log	every problem: source, row, column, bad value, error type, action, message
sec_roles, sec_role_permissions	the 5 roles and the permission matrix

Table	Purpose
sec_users	users, role, department (heads), PBKDF2 password hash + salt, lock-out fields
audit_log	logins, queries (allowed and refused) and changes recorded by triggers

Views: `v_usage_cube` (OLAP cube), `v_dim_student_masked`, `v_dept_book_loans`, `v_dept_digital_usage`, `v_dept_room_bookings`, `v_dept_monthly_summary`, `v_audit_report`.

8. ETL design

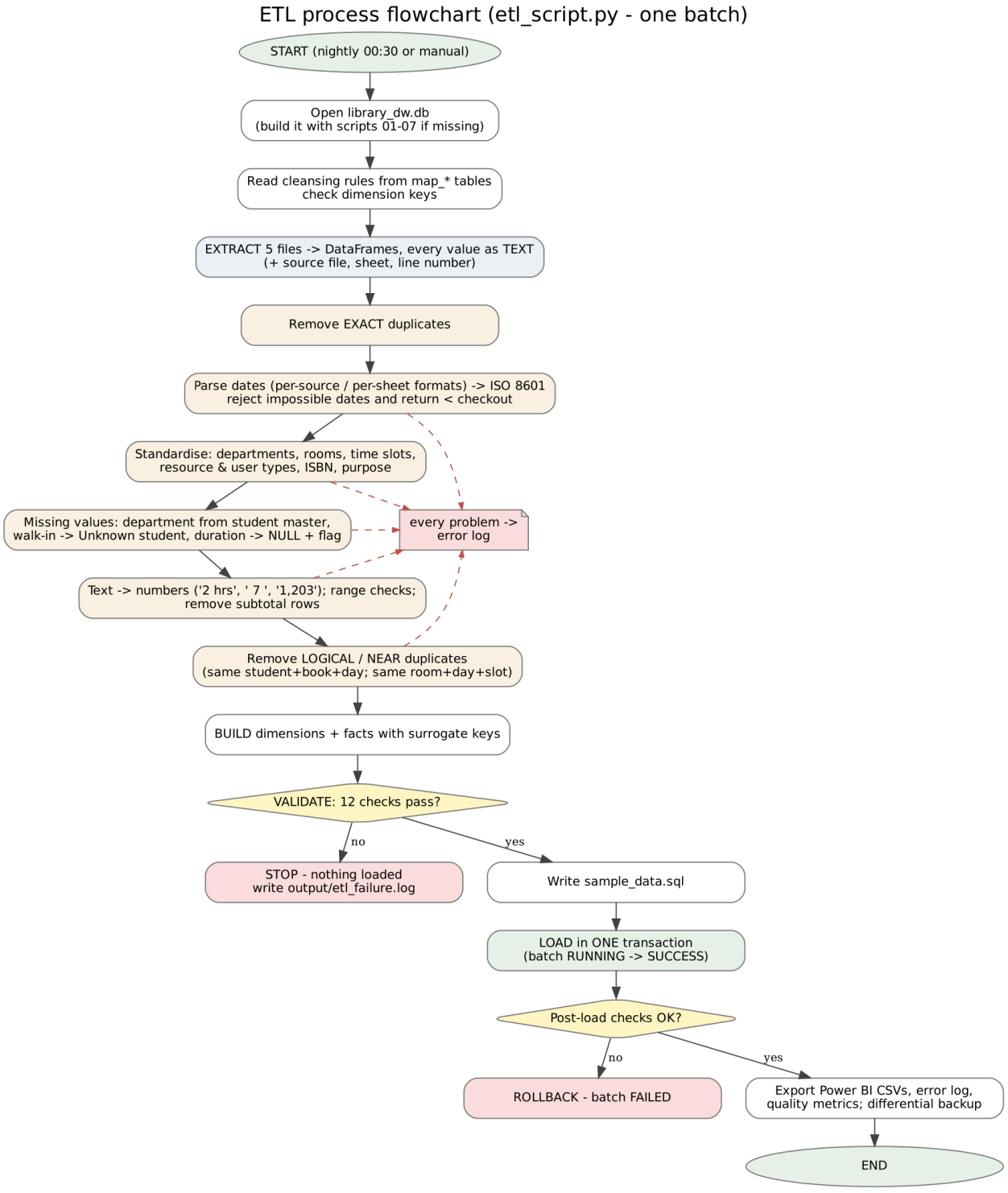


Figure 4 - ETL process flowchart

8.1 Extraction plan

Source	How it is read	Special handling
book_transactions.csv	`pandas.read_csv(dtype=str)`	empty cells → NULL; "N/A" kept as text so it can be logged

Source	How it is read	Special handling
digital_usage.xlsx	`pandas.read_excel(sheet_name=None)` - **all sheets** stacked	sheet name stored (needed for its date format)
room_bookings.csv	`read_csv(dtype=str)`	
student_master.csv, book_catalog.csv	`read_csv(dtype=str)`	clean reference data

8.2 Transformation logic

The 30 cleansing rules are listed in `04_ETL_Files/transformation_rules.xlsx`; the main ones:

- **Dates → ISO 8601.** Each source has a list of known formats that are tried in order. The Excel file uses one format *per sheet*; because "2024-05-03" means 5 March on the 2024 sheet (year-day-month) but 3 May elsewhere, the sheet's own rule is used and the year is checked.
- **Names → codes** through the `map_*` tables (lower-case, spaces trimmed): departments, time slots; dictionaries for resource types, user types and purposes; rooms by keeping the digits ("Room 101" → "R101").
- **Text → numbers**: spaces, commas, decimals and units removed ("2 hrs", " 7 ", "1,203").
- **Missing values**: department filled from the student master; walk-ins → Unknown student; missing reading time kept as NULL + flag (no invented data); return date missing = not returned yet.
- **Duplicates** at three levels: *exact* (identical row), *logical* (same student + book + day, new ID), *near* (same room + day + slot after standardising - catches "Morning"/"8AM-10AM", "R-101"/"Room101").
- **Subtotal rows** typed into the Excel sheets (QUARTER TOTAL) are removed.

8.3 Loading strategy

- **Full load (current)**: one batch replaces staging and warehouse contents inside **one transaction**; at ~9,000 rows it takes about one second. Dimensions first, then facts.
- **Incremental load (designed for growth)**: the facts carry `etl_batch_id`, and `transaction_id` / `booking_id` are UNIQUE. An incremental batch would extract only source rows newer than the last loaded date (*watermark* stored in `etl_batch_log`), add new dimension members with `INSERT OR IGNORE` and insert new facts - the UNIQUE keys make a repeated run harmless.

8.4 Error handling

Situation	What happens
a bad value	logged in `etl_error_log` with action REJECTED / CORRECTED / DEFAULTED; the run continues
a validation check fails (12 checks)	the batch stops **before** loading; nothing changes; message in `output/etl_failure.log`
an SQL error while loading	the transaction is rolled back; batch marked FAILED
a mismatch after loading	post-load row counts and foreign-key check raise an error

8.5 Data refresh schedule

Frequency	Job
daily 00:30	`run_nightly_etl.bat`: ETL + differential backup
every 4 h (08:00-20:00)	incremental backup

Frequency	Job
weekly (Sunday 01:00)	encrypted full backup
monthly (1st working day)	Director reviews the dashboards and the error-log summary; data-quality meeting

9. Indexes and performance

SQLite indexes primary keys and UNIQUE columns automatically but **not** foreign keys, so script 05 adds:

Index	Columns	Used by
idx_date_hierarchy	year, quarter, month, day	drill-down (Q6)
idx_date_week, idx_date_month	week_start_date; month_label	roll-up (Q7), monthly trends (Q1)
idx_loan_date, idx_loan_dept_date	checkout date; department + date	trends, department ranking, loan duration (Q1, Q2, Q5)
idx_loan_book	book_key	top 10 books (Q4)
idx_dig_date_dept, idx_dig_dept_res, idx_dig_resource	date+dept; dept+resource; resource	dicing (Q9), pivot (Q10), top journals (Q4)
idx_room_date_slot	date + time slot	peak times heat map (Q3)
idx_student_id_current	student_id, is_current	ETL key lookups (SCD)

All 31 queries run in well under a second.

10. Security, backup and OLAP (summary)

- **OLAP cube**: `v_usage_cube` (date × department × service); hierarchies Year → Quarter → Month → Day and Day → Week → Month in `dim_date`. Demonstrated in `olap_operations_demo.pdf`.
- **Security**: 5 roles; analysts read-only with masked StudentID; department heads see only their own department; passwords hashed; lock-out after 3 failures; every login and query in `audit_log`. Details: `RBAC_policy.pdf`, `security_implementation.pdf`.
- **Backup**: weekly encrypted full + nightly differential + 4-hourly incremental, SHA-256 checksums, automatic restore test. Details: `backup_recovery_plan.pdf`.

11. Scalability

Change	How the design copes
New library branch next year	add `dim_branch` (branch_key, name, campus) and a `branch_key` column to the three facts (existing rows → "Main library"); rooms get a branch attribute; dashboards gain a Branch slicer. No existing table is redesigned.
E-book reading-time tracking	`duration_minutes` already exists in `fact_digital_usage`; for detailed sessions add `fact_reading_session` (one row per session: start/end, pages) sharing `dim_date`, `dim_resource`, `dim_user_type` - another star in the galaxy.
More data (10× - 100×)	SQLite handles many GB; switch the ETL to incremental loads; archive staging rows older than one year.
Many simultaneous users or web access	migrate to MySQL / PostgreSQL: the model is standard SQL; scripts change only in data types and the security layer becomes database GRANTs.

12. Answers to the stakeholder questions

#	Question	Short answer
1	Walk us through your ETL pipeline. How do you handle failures?	Extract to text staging → transform (30 rules) → 12 validation checks → load in one transaction → post-load checks. A failed check stops the batch before anything changes; an SQL error rolls everything back and marks the batch FAILED. Every bad value is logged.
2	Why a star (galaxy) schema and not snowflake?	Fewer joins = faster and simpler for users and Power BI; dimensions are small, so normalising them saves nothing. Three stars share conformed dimensions.
3	How do you handle incremental loads?	Batch IDs + watermark in `etl_batch_log`, UNIQUE source IDs, `INSERT OR IGNORE` for new dimension members (section 8.3). Today a full reload takes ~1 second.
4	Which indexes and why?	Date hierarchy, week/month, fact foreign keys used in joins and filters (section 9).
5	A new branch next year?	Add `dim_branch` + `branch_key` (section 11).
6	Add e-book reading-time tracking?	`duration_minutes` exists; add `fact_reading_session` for detailed sessions.
7	Most surprising insight?	Downloads per book loan doubled from 5.9 (2023) to 12.3 (2025), and the October 2024 drop of -57.3% is detected automatically - the pattern of an e-resource platform outage. Graduates keep books 17.9 days vs 12.8 for undergraduates.
8	Monthly reports: now vs before?	Before 15+ hours of manual work; now the nightly ETL refreshes everything automatically and a complete rebuild takes about 5 seconds; dashboards are always up to date.
9	How do you ensure student privacy?	Masked IDs everywhere outside admin, column-level block on `student_id`, department-only access for heads, audit log, encrypted backups; Power BI files contain no real IDs.
10	What if the room booking system changes its format?	Raw text staging never fails; new spellings = one row in `map_*`; unknown values are defaulted and logged; validation stops a broken batch before it reaches the warehouse.
11	Who can access salary information?	Nobody: the warehouse stores no salary or HR data (data minimisation). If ever added, it would be a separate restricted table readable only by `dw_admin`.
12	How quickly can you recover after a failure?	RPO ≤ 4 hours (incremental backups), RTO < 15 minutes; the measured restore of the full chain took under one second (`backup_test_results.txt`).

13. Project files and how to run

Folder	Content
01_Design	architecture, ERD, galaxy schema, ETL flowchart (PNG + Graphviz sources)
03_Database_Files	SQL scripts 01-07, `sample_data.sql`, `build_database.py`, `library_dw.db`
04_ETL_Files	`etl_script.py`, quality report, transformation rules, error log
05_Analytics_Package	`business_queries.sql` (31 queries), results, OLAP demo, bonus analytics
06_Dashboards	Power BI data, DAX measures, theme, dashboard images, guide
07_Security_Documentation	policies, `secure_access.py`, `backup_restore.py`, test results
09_Source_Data	the three messy sources + generator

```
pip install pandas openpyxl matplotlib cryptography
python run_all.py          # rebuilds everything from scratch (about 5 seconds)
```